

DocuDigit AI

From scanned forms to editable Word documents, with a human in control.

In development

Independent project · Processing, API & worker

September 2026 · Ongoing

Python

PaddleOCR

FastAPI

SQLAlchemy

DOCX

LibreOffice

01 / CONTEXT

The problem & my contribution

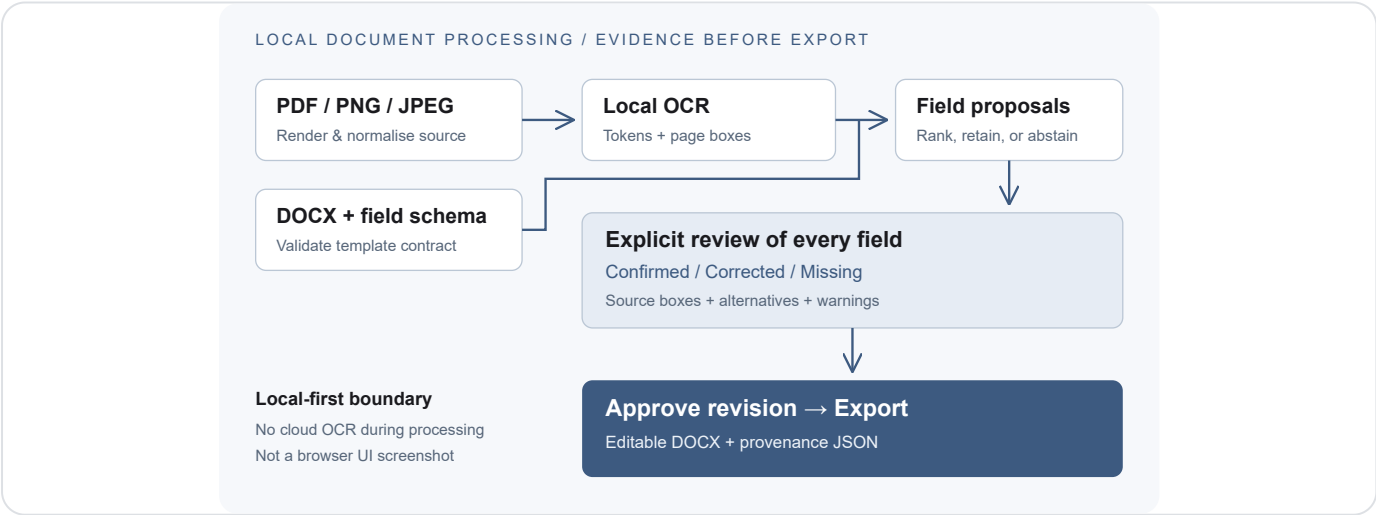
Scanned forms often need to be retyped into a standard Word template. DocuDigit AI brings local OCR, field proposals, explicit review, and document generation into one reproducible workflow. The implemented scope covers a command-line slice and a persistent API/worker; the browser review interface is the next phase.

- Process locally on a Windows CPU setup with pre-downloaded models; no cloud OCR service.
- Preserve editable Word text and simple template structure, rather than flattening the output into an image.
- Keep source evidence and require a decision for every field before export.

02 / APPROACH

From input to useful output

A configured DOCX template defines the fields. PyMuPDF renders PDFs, while PNG and JPEG inputs are normalised for local PaddleOCR. Each recognised token retains its page, text, engine score, and normalised bounding box. Deterministic mapping combines label/alias similarity, nearby geometry, and field-type checks. Weak or conflicting evidence abstains. Review records confirmed, corrected, or missing values; docxtpl produces an editable DOCX and JSON provenance record. LibreOffice supplies template/output previews.



Original workflow diagram. Human review is a required processing step, not a confidence-score shortcut.

How the pieces fit

The processing package is shared by the CLI and a versioned FastAPI service. A polling worker persists job progress and processes one job at a time. Jobs snapshot the template version and schema; private artifacts are accessed through controlled routes.

Processing	Template validation, rendering, OCR, candidate mapping, review, and export.
API & worker	Persistent jobs, revision-guarded edits, approval, retry, and idempotent export.
Storage	SQLAlchemy/Alembic records and opaque local file keys. SQLite verified; PostgreSQL integration not yet verified.

Engineering trade-offs

Evidence, not false confidence Treat rank_score as candidate ordering only. OCR engine scores and mapping heuristics are uncalibrated. Page boxes and component scores let a reviewer inspect the source instead of trusting a percentage. <i>Trade-off: Review is mandatory; handwriting and ambiguous layouts remain human decisions.</i>	Protect the approved revision Reject stale edits and recover interrupted work. Revision mismatches return HTTP 409. Startup recovery requeues processing within an attempt limit and returns interrupted exports to the approved queue. <i>Trade-off: The current design is a local, single-user worker—not a distributed production queue.</i>
--	--

Results, with boundaries

Rechecked on 5 October 2026: all 27 tests passed, including real local OCR and LibreOffice rendering. The API integration story covers template setup, scan upload, processing, review, approval, repeatable export, download, and deletion.

- Tests reject incomplete review and premature approval, retain required-missing warnings, and check stale-revision rejection.
- Native DOCX filling preserves paragraph/table shape; job deletion removes its private artifacts while retaining the template.

What this does not prove

- Evidence uses a single synthetic ten-field form family, not a multi-form OCR accuracy benchmark.
- Browser review, authentication, public-host security hardening, and live PostgreSQL validation are not implemented or verified.

Build browser evidence review against the existing API, then evaluate varied form families, abstention quality, and reviewer effort.

Source: project implementation and Phase 0–2 verification records, September 2026; tests rerun on 5 October 2026. Seven dependency/tooling warnings were reported. No personal records or runtime artifacts are published here.